TASK 9 – Implement and Test the Fare Logic

1. Objective

To implement and test the complete metro fare calculation system using C++. The program calculates fares based on travel zones and peak hours, validates smart-card balances, handles invalid inputs and verifies the correctness of transactions through test cases.

2. Fare Rules and Constants

The metro network contains three zones. The fare depends on the farthest zone travelled.

Zone	**Base Fare**
Inner	Rs. 10.00
Middle	Rs. 20.00
Outer	Rs. 30.00

Peak-hour surcharge: 20%

- Morning peak: 8:00 AM to before 10:00 AM.
- Evening peak: 5:00 PM to before 8:00 PM.
- All other hours are off-peak.

Minimum balance rule: At least Rs. 50.00 must remain on the smart card after fare deduction.

3. Fare Calculation

The fare is calculated using: $PeakFare = BaseFare \times 1.20$

$FinalFare = BaseFare + Surcharge$

For example, an Outer Zone journey during peak hours:

$30 + (30 \times 20/100) = Rs.36.00$

The implementation uses integer paise instead of floating-point values to avoid rounding errors.

4. Peak-Hour Boundary Validation

The correct peak-hour condition is:

(hour >= 8 && hour < 10) ||
(hour >= 17 && hour < 20)

Time	**Expected classification**
08:00	Peak
10:00	Off-peak
17:00	Peak
20:00	Off-peak

The starting times are inclusive, while the ending times are exclusive. This prevents inconsistent fare calculations at boundary times.

5. Smart-Card Balance Validation

A journey is approved only when:

$Balance - Fare \geq 5000 paise$

This ensures that at least Rs. 50 remains after every successful transaction.

A negative recharge is rejected because it could incorrectly reduce the card balance.

6. Alternative Concept – Table-Driven Testing

Instead of writing separate test conditions for every journey, a table of test cases can be created containing:

- Input zone.
- Travel hour.
- Expected fare.
- Expected transaction result.

A loop processes each test case and compares the actual result with the expected result.

Complexity analysis:

Parameter	**Hand-written checks**	**Table-driven testing**
Time for t tests	O(t)	O(t)
Test-case storage	O(1) per test	O(t)
Adding a boundary case	Requires another check	Add one table entry

Maintenance	Less scalable	Easier to maintain

Evaluation: Table-driven testing is more suitable because new test cases can be added without rewriting the fare calculation logic. It also makes verification systematic and repeatable.

7. Conclusion

The fare logic ensures accurate zone-based pricing, correct peak-hour surcharge calculation, safe balance validation and rejection of invalid transactions. Table-driven testing improves maintainability and makes the system easier to extend.